

댕댕댕

인증 → 반려동물 → 오픈채팅 → 배포까지 , 서비스의 현관문과 복도를 맡은
반려동물 통합 케어 플랫폼 백엔드

Java 21

Spring Boot 3.5

Spring Data JPA

WebSocket · STOMP

PostgreSQL (Supabase)

React 19

EC2 · nginx · GitHub Actions

송태현 · Backend Developer

uiop7000@naver.com · github.com/Taehyun-0502/pet_project

4 인 팀 · 2026.08.04 – 09.01
AWS EC2 배포 · 운영 · 현재 서버 운영 종료

목차 — 담당 범위 → 설계 → 문제 해결 사례 → 시스템 · 협업 → 검증 · 한계

제목이나 페이지를 누르면 이동하고, 모든 장 오른쪽 아래의 '🏠 목차' 로 돌아옵니다.

01

[PROJECT — 요약 · 서비스 · 아키텍처 · 데이터 모델](#)

커밋 110/219 · 엔드포인트 40 · 요구사항 46/69 · 14 테이블 중 6 개 소유

[03 - 06 →](#)

02

[CASE 01 — 인증 : 토큰 회전 · 재사용 감지 · 롤백 버그](#)

REQUIRES_NEW 별도 빈 · 검토한 대안과 기각 이유

[07 - 08 →](#)

03

[CASE 02 — 채팅 : REST+STOMP · AFTER COMMIT · 커넥션 풀](#)

DB 에 확정된 것만 방송 · join 비트랜잭션 + 보상

[09 - 10 →](#)

04

[SYSTEM · PROCESS — 공통 규약 · 배포 · 리뷰 검증 · 한계](#)

PR 25 · AI 리뷰 결과 검증 · Testcontainers 계획

[11 - 12 →](#)

시간이 3 분뿐이라면

이 네 장만 보셔도 됩니다. 제목을 누르면 이동합니다.

03

[요약 한 장](#)

담당 범위와 수칙

05

[아키텍처](#)

nginx · Spring · Supabase · 배포

08

[트랜잭션 롤백 버그](#)

가장 어려웠던 문제

12

[검증 · 한계](#)

확인한 것 / 부족한 것

현관문과 복도를 맡은 백엔드 — 인증 · 반려동물 · 오픈채팅 · 공통 규약 · 배포

팀 4 명이 도메인을 나눠 병렬 개발했고, 제 범위는 Git 이력과 개발 일정표로 확인되는 것만 적었습니다.

110 / 219

커밋 (병합 제외 · 본인 / 전체)

저장소 개설 · PR 25 건 리뷰 · 머지

5 개 패키지

주요 담당 영역

member · chat · pet · common · security
인증 · 채팅 · 공통 규약 · 배포 연결

40 개

REST 엔드포인트

member 13 · pet 7 · chat 20 (+ STOMP
구독 채널)

46 / 69

요구사항 담당

인증 12 · 인프라 7 · 반려동물 7 ·
오픈채팅 15 · UI/ 홈 5

1 회원 · 인증

BCrypt · JWT(HS512, 15 분) · 리프레시 토큰 회전 · 재사용 감지 · 카카오 OAuth · 기기 관리 · 원격 로그아웃 · 탈퇴

2 오픈채팅

방 · 권한 3 종 (OWNER/MANAGER/MEMBER) · 강퇴 · 위임 · 발신 REST / 수신 WebSocket(STOMP) · 읽음 · 공지 핀 · 이미지 메시지

3 반려동물 · 홈

소유자 데이터 격리 (타인 · 삭제 · 없음 전부 404) · @Version 낙관적 잠금 · 프로필 사진 Storage · 온보딩 · 홈 화면

4 공통 규약 · 인프라

ApiResponse 봉투 · 도메인별 ErrorCode · JWT 인증 필터 · 저장소 골격 · EC2 + nginx + GitHub Actions 배포

반려동물의 건강과 일상을 한 곳에 — 슷츠 · 오픈채팅 · AI 진단 · 지도 · 챗봇

팀 전체 기능은 여섯 축. 제 담당은 모든 기능이 통과하는 인증 · 회원, 오픈채팅, 반려동물, 그리고 공통 규약 · 배포입니다.

회원 · 인증

카카오 OAuth · JWT · 기기 관리

오픈채팅

STOMP 실시간 · 방 권한 · 이미지

반려동물 · 홈

펫 프로필 · 온보딩 · 홈

setWidth (릴스)

촬영→편집→발행 · 추천 (팀원)

AI 질병 진단

EfficientNet 피부 진단 (팀원)

지도 · 산책 · 챗봇

카카오맵 · 산책 트래킹 · MCP (팀원)

■ 진한 카드 = 내 담당 · 회원 1 명 = 역할 1 개 (MEMBER), 누구나 방 생성, 강아지 전용 프로필

사용자 흐름 · 모바일 퍼스트 PWA

- 가입 / 카카오 로그인 → /welcome 온보딩 (펫 등록) → 홈 (펫 카드 · 슷츠 · 채팅 · 진단 진입점) → 오픈채팅 방 검색 · 입장 · 실시간 대화
- 360px 기준 하단 탭바, 앱처럼 설치 가능. 프론트와 API 를 같은 오리진에 두어 CORS 자체를 제거



오픈채팅 · 이미지 메시지

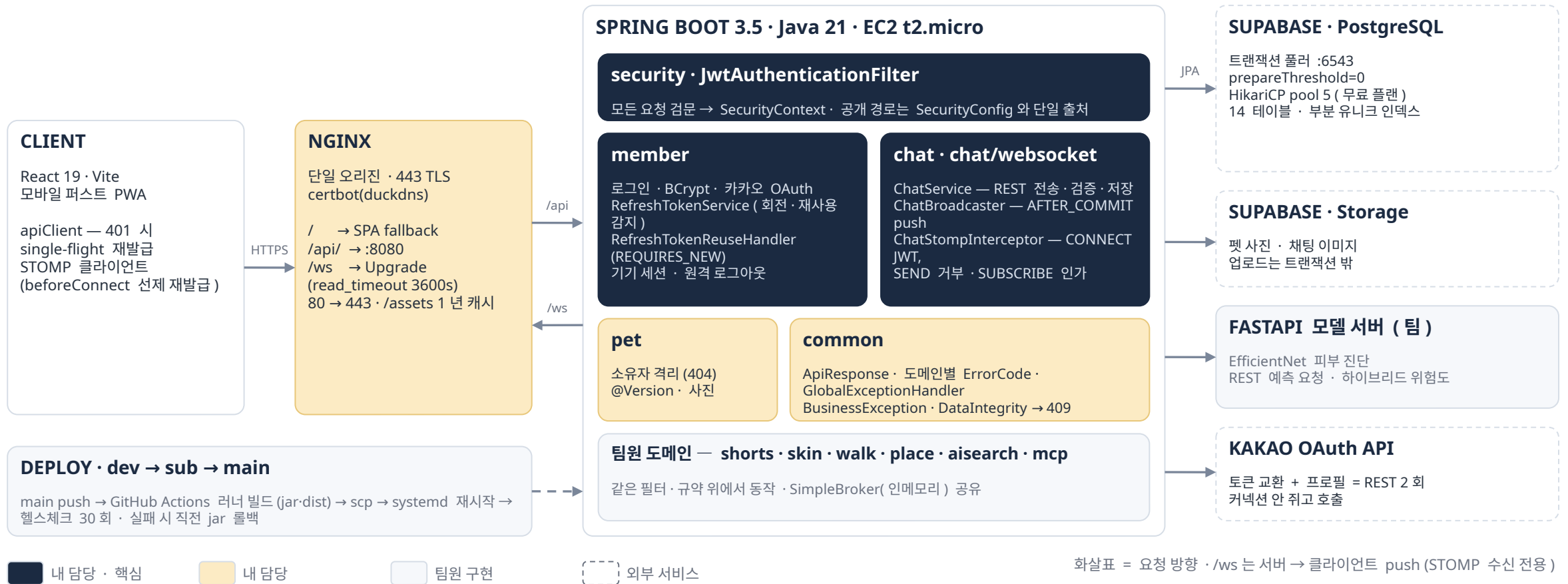


마이페이지 · 펫 정보

AWS EC2 운영 당시 화면 · 2026-09-02

nginx 단일 오리진 뒤 3 개 서버 — Spring 이 허브 , FastAPI 는 모델 전용

제 담당은 Spring 의 member · chat · pet · common · security 패키지와 nginx · systemd · GitHub Actions 배포 경로입니다 . 외부 I/O 는 트랜잭션 밖 , 방송은 커밋 후 .



pet_member 를 허브로 — 14 개 테이블 중 제가 설계 · 소유한 6 개

Long ID 참조 (JPA 연관관계 대신), 소프트 삭제 , append-only 메시지 , 동시 수정 지점에만 @Version 낙관적 잠금 .

pet_member

회원 · LOCAL/KAKAO · 소프트 삭제 ·
tokens_valid_from

AUTH

refresh_tokens

해시만 저장 · 폐기 사유 7 종 ·
session_id(기기)

PET

pet

회원 1:N · @Version · sort_order · 소프트
삭제

CHAT

chat_room

카테고리 · 정원 · 공지 핀

chat_room_member

권한 3 종 · @Version · 강퇴 이력 · 읽음
위치

chat_message

append-only · 본문 / 이미지 XOR

TEAM (읽기 참조만)

shorts · shorts_comment

숏츠 도메인 (팀원)

walk_record · place ...

산책 · 지도 도메인 (팀원)

- 채팅 메시지는 수정 · 삭제 없음 (append-only), 이미지 메시지는 content:image_url 중 정확히 하나 (CHECK)
- 활성 회원 기준 lower(email)·lower(name) 부분 유니크 인덱스 · 방당 활성 OWNER 1 명 유니크 인덱스 — 위임 경쟁을 DB 가 최종 방어

리프레시 토큰은 JWT 가 아니라 난수 — 서버 DB 가 유일한 진실이라 즉시 폐기가 가능합니다

엑세스 토큰 (15 분) 은 헤더로 , 리프레시 토큰 (14 일) 은 HttpOnly 쿠키로 . 원문은 저장하지 않고 SHA-256 해시만 보관하며 , 쓸 때마다 회전합니다 .

1 로그인

BCrypt 대조 (미존재 계정도 더미 해시 1 회 — 타이밍 균등화) → 액세스 + 리프레시 발급

2 재발급 /refresh

쿠키 토큰 해시 조회 → 만료 · 폐기 판정 → 회전 (ROTATED) → 새 토큰 , 같은 session_id 승계

3 재사용 감지

폐기된 토큰이 유예 30 초 밖에서 다시 오면 유출로 판단 → 그 회원의 활성 토큰 전부 폐기

4 기기 관리

session_id 단위 목록 · 원격 로그아웃 · 비밀번호 변경 시 전 기기 폐기 후 현재 기기만 재발급

```
// RevokedReason - 폐기 사유 7 종 . 사유가 곧 정책
ROTATED          // 정상 회전 (30 초 유예 안 재제출 허용)
LOGOUT           // 이 기기만
REUSE_DETECTED   // 유출 의심 → 전 기기 폐기
PASSWORD_CHANGED // 전 기기 폐기 , 현재 기기 재발급
REPLACED_BY_LOGIN // 재로그인 - 유령 기기 방지
DEVICE_REVOKED   // 원격 로그아웃
WITHDRAWN        // 탈퇴
```

```
// 뒤의 4 종은 " 폐기당한 쪽이 모른 채 재제출 " 이
// 보장된 정상 동작 → 재사용 감지에서 제외 (단순 401)
```

로그인된 기기

Chrome · Windows 현재 기기

로그인 2026. 9. 2. 오후 2:24 · 마지막 사용 2026. 9. 2. 오후 2:24

Chrome · Windows

로그인 2026. 9. 2. 오후 2:23 · 마지막 사용 2026. 9. 2. 오후 2:23

로그아웃

Safari · iOS

로그인 2026. 9. 1. 오전 11:24 · 마지막 사용 2026. 9. 1. 오전 11:24

로그아웃

Safari · iOS

로그인 2026. 8. 26. 오후 1:26 · 마지막 사용 2026. 9. 1. 오전 11:22

로그아웃

기기 목록 · 원격 로그아웃

배포 서버 · 단일 사용자 · 각 20 회 (2026-09-02)
내 정보 p50/p95 32/64ms · 기기 목록 49/61ms
근거 : RefreshTokenService · docs/api-spec.md 1 절

왜 유예 30 초인가

회전 커밋 후 응답이 닿기 전 재시도하거나 두 탭이 동시에 재발급하면 옛 토큰이 다시 제출될 수 있습니다 . 오탐 완화를 위해 30 초 유예를 두었으며 , 경계 시점 동시 요청은 추가 검증 과제입니다 .

재사용 감지가 '동작하는 것처럼 보이면서' 아무것도 하지 않았다

폐기 UPDATE 직후 401 예외를 던지자 같은 트랜잭션이 롤백되며 폐기까지 되돌아갔습니다. 로그만 보면 폐기가 실행된 것처럼 보여 더 찾기 어려웠습니다.

재현 : 로그인 → 재발급 (토큰 1~2) → 토큰 1 재발급 401 → 토큰 2 재발급 200 x

```
// BEFORE - RefreshTokenService (호출자 @Transactional 안)
if (token.isRevoked()) {
    refreshTokenRepository.revokeAllByMemberId(...); // 폐기 UPDATE
    throw new BusinessException(INVALID_REFRESH_TOKEN);
} // RuntimeException → 롤백 → 위 UPDATE도 함께 사라짐
```

```
// AFTER - 별도 빈 + REQUIRES_NEW: 바깥이 롤백돼도 폐기는 커밋
@Component class RefreshTokenReuseHandler {
    @Transactional(propagation = Propagation.REQUIRES_NEW)
    void revokeAllOf(Long memberId) {
        repo.revokeAllByMemberId(memberId, now(), REUSE_DETECTED);
        repo.expireRotationGraceByMember(...); // 유예 안 토큰도 무력화
        log.warn("재사용 감지 memberId={}" ...);
    }
}
```

수동 재현 결과 — 토큰 2 재발급도 401. 정상 회전 · 로그아웃 · 만료 경로도 확인. 폐기 로그만으로 DB 커밋을 판정하지 않습니다.

검토한 대안

(선택) REQUIRES_NEW 별도 빈

프록시를 거치도록 별도 빈으로 분리. 추가 커넥션이 필요하므로 풀 5 개에서 동시 호출 · 잠금 대기 검증 필요

noRollbackFor = BusinessException

한 줄이지만 다른 실패 경로의 롤백까지 사라짐 — 탈퇴 회원 재발급 시 회전 되돌리기가 깨짐

반환값으로 올려 호출자가 던지기

호출자 (MemberService) 도 @Transactional — 같은 트랜잭션이라 문제가 그대로

근거 : docs/troubleshooting.md 4 번 · member/RefreshTokenReuseHandler

보낼 땐 REST, 받을 땐 WebSocket — DB 에 확정된 것만 방송한다

전송을 REST 로 두면 인증 · 검증 · 저장 · 에러 응답이 다른 API 와 같은 길을 탑니다 . STOMP 는 수신 전용이고 , 발송은 커밋 후 이벤트 리스너가 합니다 .



```

@Component
public class ChatBroadcaster { // chat/websocket — 도메인은 이 클래스를 모른다

    @TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
    public void onMessageCreated(ChatMessageCreatedEvent e) {
        safely(" 메시지 푸시 ", () ->
            messagingTemplate.convertAndSend(roomTopic(e.roomId()),
            ChatEvent.message(e.message())));
    }

    @TransactionalEventListener(phase = AFTER_COMMIT)
    public void onMemberKicked(ChatMemberKickedEvent e) {
        safely(" 강퇴 세션 종료 ", () -> sessionRegistry.disconnectMember(e.memberId()));
    } // 롤백된 강퇴로 멀쩡한 사용자의 연결을 끊지 않기 위해
}
  
```

온라인 수신을 커밋 후 push 로 전환

커밋 순서와 ID 순서가 달라 생기던 폴링 누락을 줄이기 위해 온라인 수신을 push 로 전환했습니다 . afterId 재조회는 제공하지만 서버 중단 · 재연결 시 누락 · 중복 · 순서는 추가 검증이 필요합니다 .

SEND 는 설정이 아니라 인터셉터에서 거부

클라이언트가 SEND 로 저장 · 검증 경로를 우회하지 못하게 인터셉터에서 거부 . CONNECT JWT 인증과 SUBSCRIBE 참여 권한 검사를 적용하고 메시지 발신은 REST 로 통일했습니다 .

리뷰의 전제를 실측으로 반증

Spring 6.2 의 해당 프로브에서 AFTER_COMMIT 리스너 예외 후 201 유지를 관찰했습니다 . 모든 리스너 방식에 일반화하지 않으며 safely() 는 예외 처리와 로고를 위해 유지합니다 .

커넥션 풀 5 개라는 제약이 트랜잭션 설계를 지배했습니다

HikariCP 풀을 5 개로 설정했습니다. 외부 호출과 업로드는 DB 쓰기에서 분리하고, join 은 중복 충돌 처리와 보상을 위해 별도 흐름을 사용했습니다.

```
// ChatService.join - 의도적으로 비트랜잭션
try {
    joinedId = memberRepo.save(ChatRoomMember.join(roomId, memberId,
latestId)).getId();
} catch (DataIntegrityViolationException e) {
    // 흡수해도 되는 것은 중복 입장 (UNIQUE 위반) 뿐 - 참여 행을 확인해 구분
    if (!existsActive(roomId, memberId)) throw e; // FK-CHECK 위반은 409 로
}
// INSERT 후 재확인 : 그 사이 강퇴 · 정원초과가 커밋됐으면 내 행만 되돌린다
revertIfJoinLost(roomId, memberId, joinedId);
```

join: 중복 충돌 처리와 보상

- 중복 참여는 UNIQUE 로 제한하고, 삽입 후 상태를 재확인해 보상합니다. 제약 위반 뒤 같은 트랜잭션을 계속 쓰지 않도록 분리했습니다.
- 보상 전 서버 중단 · 동시 정원 초과는 추가 검증 과제입니다.

1 이미지 메시지 — 업로드는 밖, 저장만 짧게

Storage 업로드 후 DB 저장만 짧은 트랜잭션으로 처리합니다. 커밋 후 이벤트로 방송하며, DB 저장 실패 시 업로드 파일 정리 여부는 추가 확인 과제입니다.

2 카카오 로그인 · BCrypt 도 같은 원칙

카카오 REST 2 회와 BCrypt 를 DB 쓰기 구간에서 분리했습니다. 풀 대기 개선량은 미측정이며, 동시 로그인과 채팅 요청을 함께 실행해 검증할 계획입니다.

3 낙관적 잠금은 ' 실제로 동시 수정이 나는 지점 '에만

pet-chat_room_member 에 @Version 적용. 수동 위임 + 나가기 3 회에서는 한쪽만 성공, OWNER 1 명을 확인했습니다. 재발급은 회원 잠금 후 원격 로그아웃 상태 재확인. 반복 통합 테스트는 과제

같은 모양으로 응답하고 같은 방식으로 실패하게 — 공통 규약 오너, 그리고 main push 한 번의 배포

응답 봉투 · 에러 코드 계약 · 인증 필터 · 소유자 격리 원칙을 만들고 PR 리뷰에서 지켰습니다. 배포는 러너 빌드 → scp → systemd → 헬스체크.

```
// 응답은 항상 같은 봉투 — ApiResponse<T>
{ "success": true, "data": { ... } }
{ "success": false, "error": { "code": "CHAT_ROOM_FULL",
                              "message": "정원이 가득 찼습니다." } }

// 에러 코드는 도메인별 enum — code는 전역 유일
enum ChatErrorCode implements ErrorCode {
    ROOM_FULL(CONFLICT, "CHAT_ROOM_FULL", "..."), KICKED(FORBIDDEN, "CHAT_KICKED",
    "..."); }
```

인증은 필터에서

Haru Health의 '전역 permitAll + 수동 JWT 검증' 한계를 개선.
OncePerRequestFilter가 모든 요청을
검문, 공개 경로는 단일 출처

존재 여부를 노출하지 않는다

로그인 실패 사유 통일 + 타이밍 균등화 ·
pet 조회는 타인 · 삭제 · 없음 전부 404 ·
GlobalExceptionHandler가 원문 비노출

dev → sub →
main
개인 · 팀 통합 ·
배포

GitHub Actions

러너에서 jar ·
dist 빌드

EC2

scp → systemd
→ 헬스체크

nginx

/ 정적 · /api ·
/ws · HTTPS

왜 러너에서 빌드하나

프리티어 RAM 1GB — EC2에서 Gradle·Vite를 돌리면 OOM. 산출물만 올리고,
concurrency 그룹으로 배포 겹침 방지, 실패 시 직전 jar로 자동 롤백

단일 오리진의 보안 효과

프론트와 API가 같은 도메인 → CORS 제거, 리프레시 쿠키 SameSite=Strict 유지. HTTP
CORS와 STOMP 오리진을 같은 프로퍼티로 주입

한계

배포 파이프라인에서 테스트를 돌리지 않음 (@SpringBootTest가 팀 공용 DB에 붙는 부작용) → Testcontainers 기반 CI가 다음 과제

공통 규약과 PR 리뷰 — 재현 시나리오로 판단하고 검증 범위를 기록

AI 리뷰 결과는 그대로 반영하지 않고, 재현 · 실측과 코드 확인으로 판정했습니다. 맞으면 고치고, 틀리면 반증을 기록했습니다.

25 PR

리뷰 · 머지 담당

각자 브랜치 → PR → 리뷰 후 sub 통합 → main 배포

AI 리뷰 결과 검증

재현 및 코드 확인 후 반영

재현 결과와 코드 근거를 PR 에 기록

7 건

트러블슈팅 문서

증상 → 원인 → 검토한 대안 → 변경 → 결과

API 계약

명세 · 스키마 관리

api-spec.md 1,164 줄 · schema.sql 기준

● 코드 · 실측으로 확인한 것

- 수동 : 재사용 감지 후 토큰 2 재발급 401
- 수동 : 위임 + 나가기 3 회, 한쪽 성공 · OWNER 1 명
- 수동 : 만료 토큰 WS 재연결 후 정상 수신
- 배포 : main push → EC2 · HTTPS
- API 6 종 p95 45~78ms (각 20 회 · 단일 사용자)

● 현재 한계

- CI 테스트 미포함
- 핵심 인증 · 동시성 시나리오 실제 DB 검증 필요
- 보상 중단 · 추가 커넥션 대기 미검증
- 재연결 · 서버 중단 시 메시지 누락 · 중복 검증 필요
- SimpleBroker 인메모리 · 단일 인스턴스
- 단일 사용자 응답시간만 측정 · 부하 테스트 없음

● 다음 개선 — 검증 우선

- 1 순위 : 토큰 폐기 · join 보상 · 위임 경쟁을 실제 DB 테스트로 고정하고 CI 에 포함
- 2 순위 : 재연결 복구 · 동시 요청 · 풀 대기 측정
- 이후 : 필요 부하 확인 후 브로커 외부화 · 기능 확장

● TAKEAWAY

서비스의 현관문과 복도를 트랜잭션 경계 · 커밋 후 방송 · 제약 중심 설계로 지켰습니다

1

실패로 끝나는 요청 안에서 남겨야 할 것은 따로 커밋한다

RefreshTokenReuseHandler · REQUIRES_NEW

2

DB 에 확정된 것만 방송하고 , 도메인은 전달 수단을 모른다

ChatBroadcaster · AFTER_COMMIT · websocket → chat 한 방향

3

제약이 설계를 지배한다 — 느린 작업은 트랜잭션 밖으로

커넥션 풀 5 · join 비트랜잭션 + 보상 · 업로드 후 짧은 저장

송태현 · Backend Developer

uiop7000@naver.com · github.com/Taehyun-0502/pet_project